

Cardinality of Sets

Section 2.5
partB

The Positive Rational Numbers are Countable

- **Definition:** A *rational number* can be expressed as the ratio of two integers p and q such that $q \neq 0$.
 - $\frac{3}{4}$ is a rational number
 - $\sqrt{2}$ is not a rational number.

Example 3: Show that the positive rational numbers are countable.

Solution: The positive rational numbers are countable since they can be arranged in a sequence:

$$r_1, r_2, r_3, \dots$$

The next slide shows how this is done.

→

The Positive Rational Numbers are Countable

First row $q = 1$.
Second row $q = 2$.
etc.

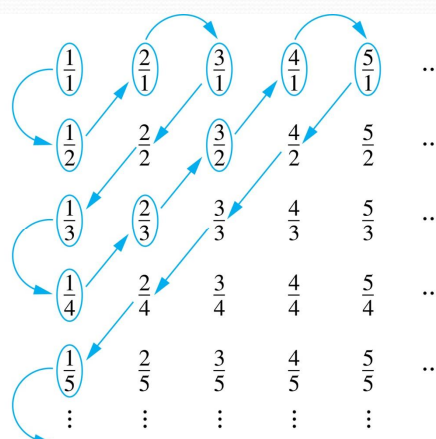
Constructing the List

First list p/q with $p + q = 2$.
Next list p/q with $p + q = 3$

And so on.

$1, \frac{1}{2}, 2, 3, \frac{1}{3}, \frac{1}{4}, \frac{2}{3}, \dots$

Terms not circled
are not listed
because they
repeat previously
listed terms



2.5 Cardinality of Sets

The set of positive rational numbers $\mathbb{Q}^+$ (another solution)

- $\forall x \in \mathbb{Q}^+, x = p/q, p, q \in \mathbb{Z}^+$
- Let $S = \{ (p, q) | p, q \in \mathbb{Z}^+ \} = \mathbb{Z}^+ \times \mathbb{Z}^+$.

■

$$\left. \begin{array}{l} |\mathbb{Q}^+| \leq |S| \\ |S| = |\mathbb{Z}^+| \\ |\mathbb{Z}^+| \leq |\mathbb{Q}^+| \end{array} \right\} \Rightarrow |\mathbb{Q}^+| = |\mathbb{Z}^+|$$

2.5 Cardinality of Sets

$$(1) |Q^+| \leq |S|$$

Suppose that $r = \frac{p}{q} \in Q^+$

$\frac{p}{q} \rightarrow (p, q)$ is injective

$$\therefore |Q^+| \leq |S|$$

5

2.5 Cardinality of Sets

(2) $|S| = |Z^+|$ **An infinite set is countable iff it is possible to list all the elements of the set in a sequence**

	1	2	3	...	p	...
1	(1,1)	(2,1)	(3,1)	...	(p,1)	...
2	(1,2)	(2,2)	(3,2)	...	(p,2)	...
3	(1,3)	(2,3)	(3,3)	...	(p,3)	...
...	...	...	...	...	...	...
q	(1,q)	(2,q)	(3,q)	...	(p,q)	...
...	...	...	...	...	...	...

$$1 + 2 + \dots + (p+q-2) = \frac{(p+q-2)(p+q-1)}{2}$$

$$n = \frac{1}{2}(p+q-2)(p+q-1) + q$$

6

$$(3) |Z^+| \leq |Q^+|$$

$$\because Z^+ \subseteq Q^+$$

$$\therefore |Z^+| \leq |Q^+|$$

Note :

- (1) There are the same number of positive rational numbers and positive integers.
- (2) The set of all rational numbers Q , positive and negative, is countable infinite.

7

Strings

Example 4: Show that the set of finite strings S over a finite alphabet A is countably infinite.

Assume an alphabetical ordering of symbols in A

Solution: Show that the strings can be listed in a sequence. First list

1. All the strings of length 0 in alphabetical order.
2. Then all the strings of length 1 in lexicographic (as in a dictionary) order.
3. Then all the strings of length 2 in lexicographic order.
4. And so on.

This implies a bijection from N to S and hence it is a countably infinite set. ◀

The set of all Java programs is countable.

Example 5: Show that the set of all Java programs is countable.

Solution: Let S be the set of strings constructed from the characters which can appear in a Java program. Use the ordering from the previous example. Take each string in turn:

- Feed the string into a Java compiler. (A Java compiler will determine if the input program is a syntactically correct Java program.)
- If the compiler says YES, this is a syntactically correct Java program, we add the program to the list.
- We move on to the next string.

In this way we construct an implied bijection from $\mathbb{N}$ to the set of Java programs. Hence, the set of Java programs is countable. ◀

2.5 Cardinality of Sets

The properties of the countable sets:

- 1) No infinite set has a smaller cardinality than a countable set.
- 2) The union of two countable sets is countable.
- 3) The union of finite number of countable sets is countable.
- 4) The union of a countable number of countable sets is countable.

3. Cantor Diagonalization Argument

---- An important technique used to construct an object which is not a member of a countable set of objects

【Theorem】 The set of real numbers between 0 and 1 is uncountable.

Proof:

$$A = \{x \mid x \in (0,1) \wedge x \in \mathbb{R}\}$$

$$\left. \begin{array}{l} (1) |Z^+| \leq |A| \\ (2) |Z^+| \neq |A| \end{array} \right\} \longrightarrow |Z^+| < |A|$$

11

$$(1) |Z^+| \leq |A|$$

$$A = \{x \mid x \in (0,1) \wedge x \in \mathbb{R}\}$$

$$B = \left\{ \frac{1}{n+1} \mid n \in \mathbb{Z}^+ \right\}$$

$$\therefore |B| = |Z^+| \quad B \subseteq A$$

$$\therefore |Z^+| \leq |A|$$

12

2.5 Cardinality of Sets

(2) $|Z^+| \neq |A|$

Assume A is countable, then let $A = \{r_1, r_2, r_3, \dots, r_n, \dots\}$

Represent each real number in the list using *its decimal expansion*.

e.g., $1/3 = .3333333\ldots$, $1/2 = .5000000\ldots = .4999999\ldots$

THE LIST....

$r_1 = 0.\mathbf{d_{11}}d_{12}d_{13}d_{14}d_{15}d_{16}\ldots$

$r_2 = 0.d_{21}\mathbf{d_{22}}d_{23}d_{24}d_{25}d_{26}\ldots$

$r_3 = 0.d_{31}d_{32}\mathbf{d_{33}}d_{34}d_{35}d_{36}\ldots$

...

Now construct the number $x = 0.x_1x_2x_3x_4x_5x_6x_7\ldots$

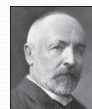
$x_i = 3$ if $d_{ii} \neq 3$

$x_i = 4$ if $d_{ii} = 3$

Then x is not equal to any number in the list.

Hence, no such list can exist and hence the interval $(0,1)$ is uncountable .

Georg Cantor
(1845-1918)



13

2.5 Cardinality of Sets

【Theorem】 The set of real numbers $R = (-\infty, +\infty)$ has the same cardinality as the set $(0,1)$.

Proof:

Let $f(x) = \tan(x)$.

$f(x)$ is a bijection from $(-\frac{\pi}{2}, \frac{\pi}{2})$ to $R = (-\infty, +\infty)$.

$$\therefore \left| \left(-\frac{\pi}{2}, \frac{\pi}{2}\right) \right| = |(0,1)| \quad \therefore |R| = |(0,1)|$$

$$|R| = \aleph_1$$

14

2.5 Cardinality of Sets

[[Example]] Suppose that $[0,1] = \{x \mid x \in \mathbb{R}, 0 \leq x \leq 1\}$. Show that the cardinality of this set is $\aleph_1$.

Proof:

$$A = [0,1] = \{x \mid x \in \mathbb{R}, 0 \leq x \leq 1\}$$

$$B = (0,1) = \{x \mid x \in \mathbb{R}, 0 < x < 1\}$$

$$(1) B \subseteq A \Rightarrow |B| \leq |A|$$

$$(2) \text{ Let } g(x) = \frac{1}{2}x + \frac{1}{4}, x \in [0,1]$$

Hence, $g(x)$ is a bijection from $[0,1]$ to $[1/4, 3/4]$.

$$\text{Thus } |A| \leq |B|$$

15

2.5 Cardinality of Sets

【Schröder-Bernstein Theorem】 If A and B are sets with $|A| \leq |B|$ and $|B| \leq |A|$ then $|A| = |B|$. In other words, if there are one-to-one functions f from A to B and g from B to A , then there is a one to one correspondence between A and B .

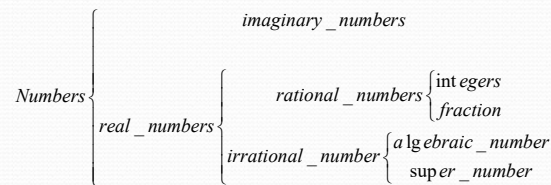
【Theorem】 The cardinality of the power set of an arbitrary set has a greater cardinality than the original arbitrary set.

The Continuum Hypothesis

The continuum hypothesis (CH) asserts that there is no cardinal number a such that $\aleph_0 < a < \aleph_1$.

16

Classification of numbers



Computability (Optional)

- **Definition:** We say that a function is **computable** if there is a computer program in some programming language that finds the values of this function. If a function is not computable we say it is **uncomputable**.
- There are uncomputable functions. We have shown that the set of Java programs is countable. Exercise 38 in the text shows that there are uncountably many different functions from a particular countably infinite set (i.e., the positive integers) to itself. Therefore (Exercise 39) there must be uncomputable functions.

Homework

Sec. 2.5 4(c,d), 28 , 36, 38

Matrices

(此节自学，作业要做)

Section 2.6

Section Summary

- Definition of a Matrix
- Matrix Arithmetic
- Transposes and Powers of Arithmetic
- Zero-One matrices

Matrices

- Matrices are useful discrete structures that can be used in many ways. For example, they are used to:
 - describe certain types of functions known as linear transformations.
 - Express which vertices of a graph are connected by edges (see Chapter 10).
- In later chapters, we will see matrices used to build models of:
 - Transportation systems.
 - Communication networks.
- Algorithms based on matrix models will be presented in later chapters.
- Here we cover the aspect of matrix arithmetic that will be needed later.

Matrix

Definition: A *matrix* is a rectangular array of numbers. A matrix with m rows and n columns is called an $m \times n$ matrix.

- The plural of matrix is *matrices*.
- A matrix with the same number of rows as columns is called *square*.
- Two matrices are *equal* if they have the same number of rows and the same number of columns and the corresponding entries in every position are equal.

3×2 matrix

$$\begin{bmatrix} 1 & 1 \\ 0 & 2 \\ 1 & 3 \end{bmatrix}$$

Notation

- Let m and n be positive integers and let

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

- The i th row of $\mathbf{A}$ is the $1 \times n$ matrix $[a_{i1}, a_{i2}, \dots, a_{in}]$. The j th column of $\mathbf{A}$ is the $m \times 1$ matrix:

$$\begin{bmatrix} a_{1j} \\ a_{2j} \\ \vdots \\ a_{mj} \end{bmatrix}$$

- The (i,j) th *element* or *entry* of $\mathbf{A}$ is the element a_{ij} . We can use $\mathbf{A} = [a_{ij}]$ to denote the matrix with its (i,j) th element equal to a_{ij} .

Matrix Arithmetic: Addition

Definition: Let $\mathbf{A} = [a_{ij}]$ and $\mathbf{B} = [b_{ij}]$ be $m \times n$ matrices. The sum of $\mathbf{A}$ and $\mathbf{B}$, denoted by $\mathbf{A} + \mathbf{B}$, is the $m \times n$ matrix that has $a_{ij} + b_{ij}$ as its (i,j) th element. In other words, $\mathbf{A} + \mathbf{B} = [a_{ij} + b_{ij}]$.

Example:

$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 2 & -3 \\ 3 & 4 & 0 \end{bmatrix} + \begin{bmatrix} 3 & 4 & -1 \\ 1 & -3 & 0 \\ -1 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 4 & -2 \\ 3 & -1 & -3 \\ 2 & 5 & 2 \end{bmatrix}$$

Note that matrices of different sizes can not be added.

Matrix Multiplication

Definition: Let $\mathbf{A}$ be an $n \times k$ matrix and $\mathbf{B}$ be a $k \times n$ matrix. The *product* of $\mathbf{A}$ and $\mathbf{B}$, denoted by $\mathbf{AB}$, is the $n \times n$ matrix that has its (i,j) th element equal to the sum of the products of the corresponding elements from the i th row of $\mathbf{A}$ and the j th column of $\mathbf{B}$. In other words, if $\mathbf{AB} = [c_{ij}]$ then $c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \dots + a_{ik}b_{kj}$.

Example:

$$\begin{bmatrix} 1 & 0 & 4 \\ 2 & 1 & 1 \\ 3 & 1 & 0 \\ 0 & 2 & 2 \end{bmatrix} \begin{bmatrix} 2 & 4 \\ 1 & 1 \\ 3 & 0 \end{bmatrix} = \begin{bmatrix} 14 & 4 \\ 8 & 9 \\ 7 & 13 \\ 8 & 2 \end{bmatrix}$$

The product of two matrices is undefined when the number of columns in the first matrix is not the same as the number of rows in the second.

Illustration of Matrix Multiplication

- The Product of $\mathbf{A} = [a_{ij}]$ and $\mathbf{B} = [b_{ij}]$

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1k} \\ a_{21} & a_{22} & \dots & a_{2k} \\ \vdots & \vdots & & \vdots \\ a_{i1} & a_{i2} & \dots & a_{ik} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mk} \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & \dots & b_{1j} & \dots & b_{1n} \\ b_{21} & b_{22} & \dots & b_{2j} & \dots & b_{2n} \\ \vdots & \vdots & & \vdots & & \vdots \\ b_{k1} & b_{k2} & \dots & b_{kj} & \dots & b_{kn} \end{bmatrix}$$

$$\mathbf{AB} = \begin{bmatrix} c_{11} & c_{12} & \dots & c_{1n} \\ c_{21} & c_{22} & \dots & c_{2n} \\ \vdots & \vdots & & \vdots \\ \vdots & \vdots & c_{ij} & \vdots \\ \vdots & \vdots & \vdots & \vdots \\ c_{m1} & c_{m2} & \dots & c_{mn} \end{bmatrix}$$

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \dots + a_{ik}b_{kj}$$

Matrix Multiplication is not Commutative

Example: Let

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix}$$

$$\mathbf{B} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$$

Does $\mathbf{AB} = \mathbf{BA}$?

Solution:

$$\mathbf{AB} = \begin{bmatrix} 2 & 2 \\ 5 & 3 \end{bmatrix}$$

$$\mathbf{BA} = \begin{bmatrix} 4 & 3 \\ 3 & 2 \end{bmatrix}$$

$$\mathbf{AB} \neq \mathbf{BA}$$

Identity Matrix and Powers of Matrices

Definition: The *identity matrix* of order n is the $m \times n$ matrix $\mathbf{I}_n = [\delta_{ij}]$, where $\delta_{ij} = 1$ if $i = j$ and $\delta_{ij} = 0$ if $i \neq j$.

$$\mathbf{I}_n = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix}$$

$$\mathbf{A}\mathbf{I}_n = \mathbf{I}_m\mathbf{A} = \mathbf{A}$$

when $\mathbf{A}$ is an $m \times n$ matrix

Powers of square matrices can be defined. When $\mathbf{A}$ is an $n \times n$ matrix, we have:

$$\mathbf{A}^0 = \mathbf{I}_n \quad \mathbf{A}^r = \underbrace{\mathbf{A}\mathbf{A}\mathbf{A}\dots\mathbf{A}}_{r \text{ times}}$$

Transposes of Matrices

Definition: Let $\mathbf{A} = [a_{ij}]$ be an $m \times n$ matrix. The *transpose* of $\mathbf{A}$, denoted by $\mathbf{A}^t$, is the $n \times m$ matrix obtained by interchanging the rows and columns of $\mathbf{A}$.

If $\mathbf{A}^t = [b_{ij}]$, then $b_{ij} = a_{ji}$ for $i = 1, 2, \dots, n$
and $j = 1, 2, \dots, m$.

The transpose of the matrix $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ is the matrix $\begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$.

Transposes of Matrices

Definition: A square matrix $\mathbf{A}$ is called symmetric if $\mathbf{A} = \mathbf{A}^t$. Thus $\mathbf{A} = [a_{ij}]$ is symmetric if $a_{ij} = a_{ji}$ for i and j with $1 \leq i \leq n$ and $1 \leq j \leq n$.

The matrix $\begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$ is square.

Square matrices do not change when their rows and columns are interchanged.

Zero-One Matrices

Definition: A matrix all of whose entries are either 0 or 1 is called a *zero-one matrix*. (These will be used in Chapters 9 and 10.)

Algorithms operating on discrete structures represented by zero-one matrices are based on Boolean arithmetic defined by the following Boolean operations:

$$b_1 \wedge b_2 = \begin{cases} 1 & \text{if } b_1 = b_2 = 1 \\ 0 & \text{otherwise} \end{cases} \quad b_1 \vee b_2 = \begin{cases} 1 & \text{if } b_1 = 1 \text{ or } b_2 = 1 \\ 0 & \text{otherwise} \end{cases}$$

Zero-One Matrices

Definition: Let $\mathbf{A} = [a_{ij}]$ and $\mathbf{B} = [b_{ij}]$ be an $m \times n$ zero-one matrices.

- The *join* of $\mathbf{A}$ and $\mathbf{B}$ is the zero-one matrix with (i,j) th entry $a_{ij} \vee b_{ij}$. The *join* of $\mathbf{A}$ and $\mathbf{B}$ is denoted by $\mathbf{A} \vee \mathbf{B}$.
- The *meet* of $\mathbf{A}$ and $\mathbf{B}$ is the zero-one matrix with (i,j) th entry $a_{ij} \wedge b_{ij}$. The *meet* of $\mathbf{A}$ and $\mathbf{B}$ is denoted by $\mathbf{A} \wedge \mathbf{B}$.

Joins and Meets of Zero-One Matrices

Example: Find the join and meet of the zero-one matrices

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix}.$$

Solution: The join of $\mathbf{A}$ and $\mathbf{B}$ is

$$\mathbf{A} \vee \mathbf{B} = \begin{bmatrix} 1 \vee 0 & 0 \vee 1 & 1 \vee 0 \\ 0 \vee 1 & 1 \vee 1 & 0 \vee 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix}.$$

The meet of $\mathbf{A}$ and $\mathbf{B}$ is

$$\mathbf{A} \wedge \mathbf{B} = \begin{bmatrix} 1 \wedge 0 & 0 \wedge 1 & 1 \wedge 0 \\ 0 \wedge 1 & 1 \wedge 1 & 0 \wedge 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}.$$

Boolean Product of Zero-One Matrices

Definition: Let $\mathbf{A} = [a_{ij}]$ be an $m \times k$ zero-one matrix and $\mathbf{B} = [b_{ij}]$ be a $k \times n$ zero-one matrix. The *Boolean product* of $\mathbf{A}$ and $\mathbf{B}$, denoted by $\mathbf{A} \odot \mathbf{B}$, is the $m \times n$ zero-one matrix with (i,j) th entry

$$c_{ij} = (a_{i1} \wedge b_{1j}) \vee (a_{i2} \wedge b_{2j}) \vee \dots \vee (a_{ik} \wedge b_{kj}).$$

Example: Find the Boolean product of $\mathbf{A}$ and $\mathbf{B}$, where

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}.$$

Continued on next slide $\rightarrow$

Boolean Product of Zero-One Matrices

Solution: The Boolean product $\mathbf{A} \odot \mathbf{B}$ is given by

$$\begin{aligned} \mathbf{A} \odot \mathbf{B} &= \begin{bmatrix} (1 \wedge 1) \vee (0 \wedge 0) & (1 \wedge 1) \vee (0 \wedge 1) & (1 \wedge 0) \vee (0 \wedge 1) \\ (0 \wedge 1) \vee (1 \wedge 0) & (0 \wedge 1) \vee (1 \wedge 1) & (0 \wedge 0) \vee (1 \wedge 1) \\ (1 \wedge 1) \vee (0 \wedge 0) & (1 \wedge 1) \vee (0 \wedge 1) & (1 \wedge 0) \vee (0 \wedge 1) \end{bmatrix} \\ &= \begin{bmatrix} 1 \vee 0 & 1 \vee 0 & 0 \vee 0 \\ 0 \vee 0 & 0 \vee 1 & 0 \vee 1 \\ 1 \vee 0 & 1 \vee 0 & 0 \vee 0 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix}. \end{aligned}$$

Boolean Powers of Zero-One Matrices

Definition: Let $\mathbf{A}$ be a square zero-one matrix and let r be a positive integer. The r th Boolean power of $\mathbf{A}$ is the Boolean product of r factors of $\mathbf{A}$, denoted by $\mathbf{A}^{[r]}$. Hence,

$$\mathbf{A}^{[r]} = \underbrace{\mathbf{A} \odot \mathbf{A} \odot \dots \odot \mathbf{A}}_{r \text{ times}}.$$

We define $\mathbf{A}^{[0]}$ to be $\mathbf{I}_n$.

(The Boolean product is well defined because the Boolean product of matrices is associative.)

Boolean Powers of Zero-One Matrices

Example: Let $\mathbf{A} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \end{bmatrix}$.

Find $\mathbf{A}^n$ for all positive integers n .

Solution:

$$\mathbf{A}^{[2]} = \mathbf{A} \odot \mathbf{A} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix} \quad \mathbf{A}^{[3]} = \mathbf{A}^{[2]} \odot \mathbf{A} = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

$$\mathbf{A}^{[4]} = \mathbf{A}^{[3]} \odot \mathbf{A} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$\mathbf{A}^{[5]} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad \mathbf{A}^{[n]} = \mathbf{A}^{[5]} \quad \text{for all positive integers } n \text{ with } n \geq 5.$$

Homework

Sec. 2.6 27

Algorithms

Chapter 3

Chapter Summary

- Algorithms
 - Example Algorithms
 - Algorithmic Paradigms
- Growth of Functions
 - Big-O and other Notation
- Complexity of Algorithms

Algorithms

Section 3.1

Section Summary

- Properties of Algorithms
- Algorithms for Searching and Sorting
- Greedy Algorithms
- Halting Problem

Problems and Algorithms

- In many domains there are key general problems that ask for output with specific properties when given valid input.
- The first step is to precisely state the problem, using the appropriate structures to specify the input and the desired output.
- We then solve the general problem by specifying the steps of a procedure that takes a valid input and produces the desired output. This procedure is called an *algorithm*.



Algorithms

Abu Ja'far Mohammed Ibin Musa Al-Khowarizmi
(780-850)

Definition: An *algorithm* is a finite set of precise instructions for performing a computation or for solving a problem.

Example: Describe an algorithm for finding the maximum value in a finite sequence of integers.

Solution: Perform the following steps:

1. Set the temporary maximum equal to the first integer in the sequence.
2. Compare the next integer in the sequence to the temporary maximum.
 - If it is larger than the temporary maximum, set the temporary maximum equal to this integer.
3. Repeat the previous step if there are more integers. If not, stop.
4. When the algorithm terminates, the temporary maximum is the largest integer in the sequence.

Specifying Algorithms

- Algorithms can be specified in different ways. Their steps can be described in English or in *pseudocode*.
- Pseudocode is an intermediate step between an English language description of the steps and a coding of these steps using a programming language.
- The form of pseudocode we use is specified in Appendix 3. It uses some of the structures found in popular languages such as C++ and Java.
- Programmers can use the description of an algorithm in pseudocode to construct a program in a particular language.
- Pseudocode helps us analyze the time required to solve a problem using an algorithm, independent of the actual programming language used to implement algorithm.

Properties of Algorithms

- *Input*: An algorithm has input values from a specified set.
- *Output*: From the input values, the algorithm produces the output values from a specified set. The output values are the solution.
- *Correctness*: An algorithm should produce the correct output values for each set of input values.
- *Finiteness* : An algorithm should produce the output after a finite number of steps for any input.
- *Effectiveness* : It must be possible to perform each step of the algorithm correctly and in a finite amount of time.
- *Generality* : The algorithm should work for all problems of the desired form.

3.1 Algorithm

Finding the Maximum Element in a Finite Sequence

【Example 1】 Describe an algorithm for finding the maximum (largest) value in a finite sequence of integers.

Solution :

$$a_1, a_2, a_3, \dots, a_n$$

1. Set the temporary maximum equal to the first integer in the sequence.
2. Compare the next integer in the sequence to the temporary maximum, and if it is larger than the temporary maximum, set the temporary maximum equal to this integer.
3. Repeat the previous step if there are more integers in the sequence.
4. Stop when there are no integers left in the sequence. The temporary maximum at this point is the largest integer in the sequence.

Finding the Maximum Element in a Finite Sequence

- The algorithm in pseudocode:

```
procedure max( $a_1, a_2, \dots, a_n$ : integers)
   $max := a_1$ 
  for  $i := 2$  to  $n$ 
    if  $max < a_i$  then  $max := a_i$ 
  return  $max$ { $max$  is the largest element}
```

- Does this algorithm have all the properties listed on the previous slide?

Some Example Algorithm Problems

- Three classes of problems will be studied in this section.
 1. *Searching Problems*: finding the position of a particular element in a list.
 2. *Sorting problems*: putting the elements of a list into increasing order.
 3. *Optimization Problems*: determining the optimal value (maximum or minimum) of a particular quantity over all possible inputs.

Searching Problems

Definition: The general *searching problem* is to locate an element x in the list of distinct elements $a_1, a_2, \dots, a_n$, or determine that it is not in the list.

- The solution to a searching problem is the location of the term in the list that equals x (that is, i is the solution if $x = a_i$) or 0 if x is not in the list.
- For example, a library might want to check to see if a patron is on a list of those with overdue books before allowing him/her to checkout another book.
- We will study two different searching algorithms; linear search and binary search.

Linear Search Algorithm

- The linear search algorithm locates an item in a list by examining elements in the sequence one at a time, starting at the beginning.
 - First compare x with a_1 . If they are equal, return the position 1.
 - If not, try a_2 . If $x = a_2$, return the position 2.
 - Keep going, and if no match is found when the entire list is scanned, return 0.

```

procedure linear search( $x$ :integer,
                         $a_1, a_2, \dots, a_n$ : distinct integers)
 $i := 1$ 
while ( $i \leq n$  and  $x \neq a_i$ )
     $i := i + 1$ 
if  $i \leq n$  then location :=  $i$ 
else location := 0
return location {location is the subscript of the term that
                  equals  $x$ , or is 0 if  $x$  is not found}
  
```

Binary Search

- Assume the input is a list of items in increasing order.
- The algorithm begins by comparing the element to be found with the middle element.
 - If the middle element is lower, the search proceeds with the upper half of the list.
 - If it is not lower, the search proceeds with the lower half of the list (through the middle position).
- Repeat this process until we have a list of size 1.
 - If the element we are looking for is equal to the element in the list, the position is returned.
 - Otherwise, 0 is returned to indicate that the element was not found.
- In Section 3.3, we show that the binary search algorithm is much more efficient than linear search.

Binary Search

- Here is a description of the binary search algorithm in pseudocode.

```

procedure binary search( $x$ : integer,  $a_1, a_2, \dots, a_n$ : increasing integers)
   $i := 1$  { $i$  is the left endpoint of interval}
   $j := n$  { $j$  is right endpoint of interval}
  while  $i < j$ 
     $m := \lfloor (i + j) / 2 \rfloor$ 
    if  $x > a_m$  then  $i := m + 1$ 
    else  $j := m$ 
  if  $x = a_i$  then  $location := i$ 
  else  $location := 0$ 
  return  $location$  { $location$  is the subscript  $i$  of the term  $a_i$  equal to  $x$ ,
    or 0 if  $x$  is not found}
  
```

Binary Search

Example: The steps taken by a binary search for 19 in the list:

1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

1. The list has 16 elements, so the midpoint is 8. The value in the 8th position is 10. Since $19 > 10$, further search is restricted to positions 9 through 16.
1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22
2. The midpoint of the list (positions 9 through 16) is now the 12th position with a value of 16. Since $19 > 16$, further search is restricted to the 13th position and above.
1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22
3. The midpoint of the current list is now the 14th position with a value of 19. Since $19 = 19$, further search is restricted to the portion from the 13th through the 14th positions.
1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22
4. The midpoint of the current list is now the 13th position with a value of 18. Since $19 > 18$, search is restricted to the portion from the 18th position through the 18th.
1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22
5. Now the list has a single element and the loop ends. Since $19 = 19$, the location 16 is returned.

Sorting

- To *sort* the elements of a list is to put them in increasing order (numerical order, alphabetic, and so on).
- Sorting is an important problem because:
 - A nontrivial percentage of all computing resources are devoted to sorting different kinds of lists, especially applications involving large databases of information that need to be presented in a particular order (e.g., by customer, part number etc.).
 - An amazing number of fundamentally different algorithms have been invented for sorting. Their relative advantages and disadvantages have been studied extensively.
 - Sorting algorithms are useful to illustrate the basic notions of computer science.
- A variety of sorting algorithms are studied in this book; binary, insertion, bubble, selection, merge, quick, and tournament.
- In Section 3.3, we'll study the amount of time required to sort a list using the sorting algorithms covered in this section.

Bubble Sort

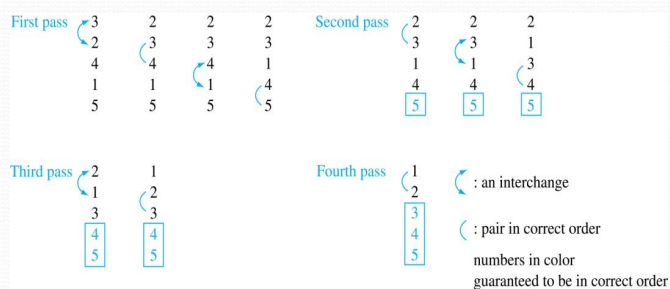
- *Bubble sort* makes multiple passes through a list. Every pair of elements that are found to be out of order are interchanged.

```

procedure bubblesort( $a_1, \dots, a_n$ : real numbers
                    with  $n \geq 2$ )
  for  $i := 1$  to  $n - 1$ 
    for  $j := 1$  to  $n - i$ 
      if  $a_j > a_{j+1}$  then interchange  $a_j$  and  $a_{j+1}$ 
  { $a_1, \dots, a_n$  is now in increasing order}
  
```

Bubble Sort

Example: Show the steps of bubble sort with 3 2 4 1 5



- At the first pass the largest element has been put into the correct position
- At the end of the second pass, the 2nd largest element has been put into the correct position.
- In each subsequent pass, an additional element is put in the correct position.

Insertion Sort

- *Insertion sort* begins with the 2nd element. It compares the 2nd element with the 1st and puts it before the first if it is not larger.

- Next the 3rd element is put into the correct position among the first 3 elements.
- In each subsequent pass, the $n+1^{\text{st}}$ element is put into its correct position among the first $n+1$ elements.
- Linear search is used to find the correct position.

```

procedure insertion sort
  ( $a_1, \dots, a_n$ :
    real numbers with  $n \geq 2$ )
  for  $j := 2$  to  $n$ 
     $i := 1$ 
    while  $a_j > a_i$ 
       $i := i + 1$ 
     $m := a_j$ 
    for  $k := 0$  to  $j - i - 1$ 
       $a_{j-k} := a_{j-k-1}$ 
     $a_i := m$ 
  {Now  $a_1, \dots, a_n$  is in increasing order}
  
```

Insertion Sort

Example: Show all the steps of insertion sort with the input: 3 2 4 1 5

- 2 3 4 1 5 (*first two positions are interchanged*)
- 2 3 4 1 5 (*third element remains in its position*)
- 1 2 3 4 5 (*fourth is placed at beginning*)
- 1 2 3 4 5 (*fifth element remains in its position*)

Greedy Algorithms



- *Optimization problems* minimize or maximize some parameter over all possible inputs.
- Among the many optimization problems we will study are:
 - Finding a route between two cities with the smallest total mileage.
 - Determining how to encode messages using the fewest possible bits.
 - Finding the fiber links between network nodes using the least amount of fiber.
- Optimization problems can often be solved using a *greedy algorithm*, which makes the “best” choice at each step. Making the “best choice” at each step does not necessarily produce an optimal solution to the overall problem, but in many instances, it does.
- After specifying what the “best choice” at each step is, we try to prove that this approach always produces an optimal solution, or find a counterexample to show that it does not.
- The greedy approach to solving problems is an example of an algorithmic paradigm, which is a general approach for designing an algorithm. We return to algorithmic paradigms in Section 3.3.



Greedy Algorithms: Making Change

Example: Design a greedy algorithm for making change (in U.S. money) of n cents with the following coins: quarters (25 cents), dimes (10 cents), nickels (5 cents), and pennies (1 cent), using the least total number of coins.

Idea: At each step choose the coin with the largest possible value that does not exceed the amount of change left.

1. If $n = 67$ cents, first choose a quarter leaving $67 - 25 = 42$ cents. Then choose another quarter leaving $42 - 25 = 17$ cents
2. Then choose 1 dime, leaving $17 - 10 = 7$ cents.
3. Choose 1 nickel, leaving $7 - 5 = 2$ cents.
4. Choose a penny, leaving one cent. Choose another penny leaving 0 cents.

Greedy Change-Making Algorithm

Solution: Greedy change-making algorithm for n cents. The algorithm works with any coin denominations $c_1, c_2, \dots, c_r$.

```

procedure change( $c_1, c_2, \dots, c_r$ : values of coins, where  $c_1 > c_2 > \dots > c_r$ ;
 $n$ : a positive integer)
for  $i := 1$  to  $r$ 
   $d_i := 0$  [ $d_i$  counts the coins of denomination  $c_i$ ]
  while  $n \geq c_i$ 
     $d_i := d_i + 1$  [add a coin of denomination  $c_i$ ]
     $n = n - c_i$ 
  [ $d_i$  counts the coins  $c_i$ ]
  
```

- For the example of U.S. currency, we may have quarters, dimes, nickels and pennies, with $c_1 = 25$, $c_2 = 10$, $c_3 = 5$, and $c_4 = 1$.

Proving Optimality for U.S. Coins

- Show that the change making algorithm for U.S. coins is optimal.

Lemma 1: If n is a positive integer, then n cents in change using quarters, dimes, nickels, and pennies, using the fewest coins possible has at most 2 dimes, 1 nickel, 4 pennies, and cannot have 2 dimes and a nickel. The total amount of change in dimes, nickels, and pennies must not exceed 24 cents.

Proof: By contradiction

- If we had 3 dimes, we could replace them with a quarter and a nickel.
- If we had 2 nickels, we could replace them with 1 dime.
- If we had 5 pennies, we could replace them with a nickel.
- If we had 2 dimes and 1 nickel, we could replace them with a quarter.
- The allowable combinations, have a maximum value of 24 cents; 2 dimes and 4 pennies.

Proving Optimality for U.S. Coins

Theorem: The greedy change-making algorithm for U.S. coins produces change using the fewest coins possible.

Proof: By contradiction.

1. Assume there is a positive integer n such that change can be made for n cents using quarters, dimes, nickels, and pennies, with a fewer total number of coins than given by the algorithm.
2. Then, $q' \leq q$ where q' is the number of quarters used in this optimal way and q is the number of quarters in the greedy algorithm's solution. But this is not possible by Lemma 1, since the value of the coins other than quarters can not be greater than 24 cents.
3. Similarly, by Lemma 1, the two algorithms must have the same number of dimes, nickels, and quarters.

Greedy Change-Making Algorithm

- Optimality depends on the denominations available.
- For U.S. coins, optimality still holds if we add half dollar coins (50 cents) and dollar coins (100 cents).
- But if we allow only quarters (25 cents), dimes (10 cents), and pennies (1 cent), the algorithm no longer produces the minimum number of coins.
 - Consider the example of 31 cents. The optimal number of coins is 4, i.e., 3 dimes and 1 penny. What does the algorithm output?

A failure of the greedy algorithm

- In some (fictional) monetary system, “Asiarons” come in 1 Asiaron, 7 Asiaron, and 10 Asiaron coins
- Using a greedy algorithm to count out 15 Asiarons, you would get
 - A 10 Asiaron piece
 - Five 1 Asiaron pieces, for a total of 15 Asiarons
 - This requires six coins
- A better solution would be to use two 7 Asiaron pieces and one 1 Asiaron piece
 - This only requires three coins
- The greedy algorithm results in a solution, but not in an optimal solution

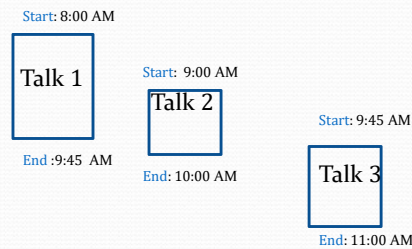
Greedy Scheduling

Example: We have a group of proposed talks with start and end times. Construct a greedy algorithm to schedule as many as possible in a lecture hall, under the following assumptions:

- When a talk starts, it continues till the end.
- No two talks can occur at the same time.
- A talk can begin at the same time that another ends.
- Once we have selected some of the talks, we cannot add a talk which is incompatible with those already selected because it overlaps at least one of these previously selected talks.
- How should we make the “best choice” at each step of the algorithm? That is, which talk do we pick ?
 - The talk that starts earliest among those compatible with already chosen talks?
 - The talk that is shortest among those already compatible?
 - The talk that ends earliest among those compatible with already chosen talks?

Greedy Scheduling

- Picking the shortest talk doesn't work.



- Can you find a counterexample here?
- But picking the one that ends soonest does work. The algorithm is specified on the next page.

Greedy Scheduling algorithm

Solution: At each step, choose the talks with the earliest ending time among the talks compatible with those selected.

```

procedure schedule( $s_1 \leq s_2 \leq \dots \leq s_n$  : start times,  $e_1 \leq e_2 \leq \dots \leq e_n$  :
    end times)
    sort talks by finish time and reorder so that  $e_1 \leq e_2 \leq \dots \leq e_n$ 
     $S := \emptyset$ 
    for  $j := 1$  to  $n$ 
        if talk  $j$  is compatible with  $S$  then
             $S := S \cup \{\text{talk } j\}$ 
    return  $S$  [  $S$  is the set of talks scheduled ]
  
```

- Will be proven correct by induction in Chapter 5.

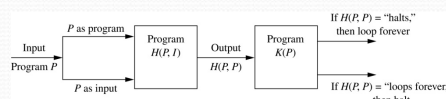
Halting Problem

Example: Can we develop a procedure that takes as input a computer program along with its input and determines whether the program will eventually halt with that input.

- **Solution:** Proof by contradiction.
- Assume that there is such a procedure and call it $H(P,I)$. The procedure $H(P,I)$ takes as input a program P and the input I to P .
 - H outputs “halt” if it is the case that P will stop when run with input I .
 - Otherwise, H outputs “loops forever.”

Halting Problem

- Since a program is a string of characters, we can call $H(P,P)$. Construct a procedure $K(P)$, which works as follows.
 - If $H(P,P)$ outputs “loops forever” then $K(P)$ halts.
 - If $H(P,P)$ outputs “halt” then $K(P)$ goes into an infinite loop printing “ha” on each iteration.



Halting Problem

- Now we call K with K as input, i.e. $K(K)$.
 - If the output of $H(K,K)$ is “loops forever” then $K(K)$ halts. **A Contradiction.**
 - If the output of $H(K,K)$ is “halts” then $K(K)$ loops forever. **A Contradiction.**
- Therefore, there can not be a procedure that can decide whether or not an arbitrary program halts. The halting problem is unsolvable.

Halting Problem

```

• Bool DOES-HALT(P, I)
• {
•   if (P will stop when run with input I)    //P(I) will stop
•       return 1
•   else
•       return 0
• }
• Bool SELF-HALT(program)
• { if(DOES-HALT(program, program))
•   infinite loop
•   else
•   halt
• }
• Then how about
•                               DOES-HALT(SELF-HALT, SELF-HALT)
•

```

Homework

Sec. 3.1 2, 4